

M47 — Room-level med carts: create on Multi-Patient Control, link encounters, grouped MAR, dispense transcript

Phase: Phase 7 follow-on (post-M46, operator feature request — closes the M44 §8 deferred work) **Status:** **DONE Blocked by:** M2 (Encounter dataclass), M5 (Multi-Patient Control dashboard), M22 (Per-Patient Console), M43 (multi-patient device routes), M44 (cabinet block on per-encounter add) **Blocks:** none **Estimated effort:** 1.5 days

1. Purpose

Operator feature request (closes the M44 §8 deferred item):

"On the multi-patient control page setup a device generator for Medication Carts. Have this setup to be able to assign one or more encounters and use the medication list under the name of the patient characters to populate the medications on the cart. When a student accesses the medication in the cart under the assigned patient character from an encounter the transcript for the encounter will be updated with the cart name time, medication name and amount and if any medication was wasted and the name of the person who wasted the excess medication."

Three pieces landed in one module:

1. **Multi-Patient Control "■ Med carts" panel.** Operator types a cart label, clicks Add. The cart shows up with a QR + link dropdown. Each cart can be linked to multiple encounters; the primary encounter is fixed and the rest are toggleable chips. 2. **Cabinet bootstrap merges MARs across linked encounters.** The cart UI's characters payload now contains every linked encounter's selected personas with their MAR data, each tagged with `encounter_id` and `encounter_label` so the cart UI can render grouped sections per patient. 3. **med.dispensed events write a transcript entry to the correct encounter.** The device event payload names the `character_id`; the server finds the linked encounter that owns that persona and calls `enc.log_turn(...)` with a composed line carrying cart name, medication, amount, who dispensed, and any waste + witness.

Plus the foundational work that M44 §8 listed:

- New `ControlRoom.cart_links`: `dict[cart_sid, list[encounter_id]]` and `cart_labels`: `dict[cart_sid, str]` fields (in-memory, no schema migration).
- New routes:
- `POST /api/room/med_cart/register`
- `POST /api/room/med_cart/{sid}/link_encounter`
- `DELETE /api/room/med_cart/{sid}/link_encounter/{eid}`
- `GET /api/room/med_carts`

The cart's underlying DB row still has a single `session_id` (its primary encounter), which keeps M43's `_session_for_station` helper working — every per-station route (inject / clear / assign) already resolves correctly. The room-level "this cart serves these encounters" projection is purely in-memory state on the active `ControlRoom`.

2. Structure

Files touched:

- `portal/control_room.py`:
- `ControlRoom` gets two new fields:
- `cart_links`: `dict[str, list[str]]` — `cart station_id` → list of linked encounter ids (the FIRST entry is the primary). - `cart_labels`: `dict[str, str]` — `cart station_id` → human label. - Both default to empty dicts; the room is created with no carts.
- `portal/server.py`:
- `New_find_room_cart(room, cart_sid)` helper (sanity check).
- Four new routes (see §1 + §4 below).
- All gated by `auth.require_instructor` (mutators) or `auth.require_vault` (the GET `/api/room/med_carts` listing). - Cart registration validates every `encounter_id` in the request body before minting the device station, and rejects creation when the room has no encounters yet. - Unlinking the primary encounter returns 409 — operator must delete + recreate the cart.
- `portal/devices/routes.py`:
- **Cabinet bootstrap** (M43-extended): now reads `room.cart_links.get(station_id)` and iterates EVERY linked encounter, calling `ehr_seed.seeds_for_all_personas(enc, ehr_id= enc.ehr_id)` per encounter. Each returned character dict gets an `encounter_id + encounter_label` tag so the cart UI can render grouped per-patient sections. - POST `/api/device/{sid}/event` (extended): when `ev_type == "med.dispensed"` AND `station["device_kind"] == "cabinet"`, calls a new `_log_cart_dispense_to_transcript()` helper after the engine has persisted the event. Failures are non-fatal — logged to `stderr`, route still returns `ok=True`. - `_log_cart_dispense_to_transcript()` resolution logic: 1. Find the linked encounter whose `selected_personas` (or `patient_persona_id`) matches the payload's `character_id`. 2. Fall back to the cart's primary session if the persona can't be matched in linked encounters. 3. Compose a `{cart_label} · dispensed {med} {amount} {unit} · by {dispenser} · wasted {n} {unit} (witness: {witness})` line and call `enc.log_turn(...)` with `source= "device:{sid}"`, `source_label=cart_label`.
- `portal/templates/control_room.html`:
- New `<section class="med-carts-panel">` between the empty-state block and the existing nurse-station panel. Gated by `{% if room %}` so it only renders when a room is active. - Carries the create form (`<input id="med-cart-label"> + <button>+ Add med cart</button>`) and an empty list (`<div id="med-carts-list">`).
- `portal/static/control_room.js`:
- New `loadMedCarts()` polls `/api/room/med_carts` every 5 s.
- New `renderMedCarts(carts)` paints each cart as a card with:
- Label + `station_id` in the header.
- Linked-encounters chip list (each chip has an "x" unlink

- button; the primary encounter shows "★ primary" instead).
- "Link encounter" dropdown + button (only renders when there are unlinked encounters available).
- QR + join URL.
- New `onMedCartAction(e1)` handles link + unlink clicks.
- New `_captureEncountersForCarts(state.room)` is hooked into the existing `pollOnce()` so the cart panel's link-encounter dropdown can show encounter labels (not just ids).
- `DOMContentLoaded` handler wires the create-cart form submit.

- portal/static/control_room.css:

- New `.med-carts-panel`, `.med-cart-create-form`, `.med-cart-card`.

.med-cart-chip, .med-cart-link-row, .med-cart-card-qr styles. Green left-border accent to distinguish from the nurse-station panel's green-tinted nursing card.

No schema migration. No new dataclass file. The `cart_links + cart_labels` dicts live on the existing `ControlRoom` dataclass and reset when the room ends — matching the in-memory model used for every other multi-encounter state today.

3. Uses

3.1 Operator flow — create + link + scan

1. Operator opens /portal/room (Multi-Patient Control). The new **Med carts** panel appears between the room status bar and the encounter grid.
2. Operator types "ICU Cart A" and clicks **+ Add med cart**. The route `POST /api/room/med_cart/register` mints a device station against the first encounter (the primary), persists `cart_links[sid] = [primary_eid]` + `cart_labels[sid] = "ICU Cart A"`, and returns the `station_id` + QR. The panel refreshes and the cart appears as a card.
3. Operator wants the cart shared between Bed 1 and Bed 2. Picks Bed 2 in the "Link encounter" dropdown, clicks **+ Link**. `POST /api/room/med_cart/{sid}/link_encounter` adds the encounter to the cart's link list. Chip appears.
4. Operator scans the cart's QR with a tablet → tablet opens `/device/join?code={primary_join}&station={sid}` → cabinet bootstrap returns ALL linked encounters' patients with their MAR meds grouped per-patient.
5. Student at the bedside taps "Bed 2 · Mr. Hayes · lorazepam 2 mg" on the cart UI; the cart fires a `med.dispensed` event with `payload = {character_id: "P-014", medication: "lorazepam", amount: "2", unit: "mg", wasted: "1", wasted_witness: "RN Jane Doe", dispensed_by: "Student Bob"}`.
6. Server resolves the persona (P-014) → finds it in Bed 1's `selected_personas` → calls `enc_bed1.log_turn(...)` with the composed line: `"■ ICU Cart A · dispensed lorazepam · 2 mg · by Student Bob · wasted 1 mg (witness: RN Jane Doe)"`
7. The instructor monitoring the Bed 1 console sees the entry appear in the next 2 s transcript poll.

3.3 Resolution order — which encounter owns this dispense?

```

payload.character_id ("P-014")
↓
for eid in room.cart_links[cart_sid]:
    enc = room.encounters[eid]
    if character_id == enc.patient_persona_id or
       character_id in enc.selected_personas:
        target_enc = enc
        break
↓
fall back to room.encounters[station.session_id]  (v6 / primary)
↓
target_enc.log_turn(...)

```

```

for eid in room.cart_links[cart_sid]:
    enc = room.encounters[eid]
    if character_id == enc.patient_persona_id or
       character_id in enc.selected_personas:
        target_enc = enc
        break

```

```
if character_id == enc.patient_persona_id or
    character_id in enc.selected_personas:
    target_enc = enc
    break
```

```
target_enc = enc
break
```

```

↓
fall back to room.encounters[station.session_id] (v6 / primary)

```

```
target_enc.log_turn(...)
```

If neither match (e.g. the persona was removed from every linked encounter mid-session), the transcript write is skipped (non-fatal).

4. Functions (exported API surface)

Symbol	Where	Purpose
POST /api/room/med_cart/register	portal/server.py	Create a room-level cart. Body: {label, encounter_ids?}. Returns {station_id, label, primary_encounter_id, linked_encounter_ids, join_url, qr_svg}.
POST /api/room/med_cart/{sid}/link_encounter	same	Body: {encounter_id}. Adds the encounter to the cart's link list. Idempotent.
DELETE /api/room/med_cart/{sid}/link_encounter/{eid}	same	Removes a non-primary encounter from the link list. 409 if attempting to unlink the primary.
GET /api/room/med_carts	same	Lists every cart with its label, primary + linked encounter ids, and QR join URL.
ControlRoom.cart_links: dict[str, list[str]]	portal/control_room.py	New field.
ControlRoom.cart_labels: dict[str, str]	same	New field.
_log_cart_dispense_to_transcript(station_id, payload)	portal/devices/routes.py	Internal helper; resolves the right encounter and writes the transcript line.

5. Limitations

- **The cart's underlying device station still has a single `session_id` in the DB.** M43's `_session_for_station` resolves via that one session id — which works for per-station ops (inject/clear/assign) — but the cart's "logical owner" is the ROOM not the encounter. If a future operator deletes the primary encounter (no UI for that yet), the cart's per-station routes would break. Tracked: when we add an encounter-delete flow, also reassign every cart's primary to another linked encounter (or fail loudly).
- **The cabinet device UI itself was not modified.** It already consumes `characters: list[dict]` from the bootstrap (V6.1.6) and renders per-patient sections. M47 just ensures the array has entries from ALL linked encounters with their `encounter_id` tag. If a future cabinet UI revision wants section headers ("Bed 1 — Mr. Hayes"), it can read `c.encounter_label` from the same payload.
- **No witness validation.** The `wasted_witness` field is a free-text string the cart UI sends. No check that the named witness is actually a registered student. Operator-facing scope; v7.0 accepts the trust model.
- **No automatic refresh** of the cart's `characters` payload when an operator links/unlinks an encounter mid-session. The cabinet device must reload to pick up the new patient list. A future WS-push from the link/unlink routes could trigger a soft refresh on connected cart tablets.
- **One Cancel button only** in the encounter chip list — and it

uses 'X'. Visual room for "Confirm" if an operator accidentally unlinks a busy bed. Not added — matches the rest of the dashboard's "click = action" convention.

- **Cart capacity** — the room's MAX_STUDENT_STATIONS_PER_ROOM

(24) doesn't apply to carts since carts aren't student stations. A cart count cap could be added if abuse becomes real; today no operator has reported needing more than a handful.

6. Test status

Test file	Asserts	Status	Last run
tests/v7/test_room_med_carts.py::test_med_cart_register_creates_room_level_cart	Cart created with primary encounter + QR	PASS	2026-05-27
...::test_med_cart_register_with_explicit_encounter_ids	encounter_ids body kwarg pre-links	PASS	2026-05-27
...::test_med_cart_register_rejects_unknown_encounter	Unknown encounter id in body → 400	PASS	2026-05-27
...::test_med_cart_register_requires_active_room	No active room → 404	PASS	2026-05-27
...::test_med_cart_link_encounter_adds_to_list	Link adds encounter to the list	PASS	2026-05-27
...::test_med_cart_link_encounter_is_idempotent	Linking twice doesn't duplicate	PASS	2026-05-27
...::test_med_cart_unlink_removes_non_primary	Non-primary unlink succeeds	PASS	2026-05-27
...::test_med_cart_unlink_primary_is_409	Primary unlink rejected with hint	PASS	2026-05-27
...::test_med_cart_link_unknown_cart_404	Unknown cart id → 404	PASS	2026-05-27
...::test_med_carts_list_returns_carts_with_links	GET /api/room/med_carts shape	PASS	2026-05-27
...::test_cabinet_bootstrap_returns_characters_from_all_linked_encs	Bootstrap merges MARs from EVERY linked encounter	PASS	2026-05-27
...::test_dispense_event_writes_transcript_to_owning_encounter	med.dispensed writes to the right encounter; line carries cart name, med, amount, by, wasted, witness	PASS	2026-05-27
...::test_dispense_event_non_dispense_types_skip_transcript	alarm.injected doesn't trigger the transcript hook	PASS	2026-05-27
...::test_control_room_dashboard_includes_med_carts_panel	Dashboard renders the panel + form + list when room is active	PASS	2026-05-27

Test file	Asserts	Status	Last run
...::test_control_room_dashboard_omits_panel_when_no_room	Empty state hides the panel	PASS	2026-05-27
Full v7 suite	332 passed, 1 skipped (M20 Playwright skip — unchanged)	PASS	2026-05-27

7. Change list

Date	Author	Change	Files
2026-05-27	claude-code	Initial M47: ControlRoom gets cart_links + cart_labels; 4 new routes; cabinet bootstrap merges MARS across linked encounters via cart_links; med.dispensed event writes a transcript entry to the encounter owning the named patient; Multi-Patient Control ■ Med carts panel + JS + CSS; 15 new tests	portal/control_room.py, portal/server.py, portal/devices/routes.py, portal/templates/control_room.html, portal/static/control_room.{js,css}, tests/v7/test_room_med_carts.py (new)

8. Open questions / known issues

- **Cabinet UI section headers.** The characters payload now

carries `encounter_id + encounter_label` per patient. The cabinet device app could group meds into collapsible per-bed sections — currently it likely renders them as one flat list. Tracked as a v6 device-UI refinement.

- **Operator delete-cart flow.** Today the only way to "remove" a

cart is to end the room (everything clears). A `DELETE /api/ room/med_cart/{sid}` route could clean up the `cart_links` entry AND the device station from SQLite. Out of scope for M47.

- **WS push when cart_links change.** A connected cart tablet

won't see a new patient until the next bootstrap. A `device_assignment`-style WS push from the link/unlink routes could trigger an auto-refresh. Deferred.

- **Dispense-event idempotency.** If the same event fires twice

(e.g. retry on flaky WS), the transcript gets two entries. The device engine has its own event log but the transcript hook doesn't dedupe. Acceptable today — the device tablet UI prevents double-tap.

*Render this guide to PDF with `python docs/module_guides/render_pdfs.py.*`